

知能情報実験 III（データマイニング班）
競馬複勝予測

グループの学籍番号列挙 245725D, 245304F, 245731J, 235751J

提出日：2026 年 6 月 9 日

目次

1	本実験向け補足	3
1.1	競馬複勝予測とは	3
1.2	実験計画	3
2	実験計画と改善 (PDCA)	3
2.1	第 1 回: ベースラインモデルの構築 (Plan・Do)	3
2.2	第 2 回: 人気・単勝オッズの追加 (Do・Check)	4
2.3	第 3 回: 特徴量の見直し (Plan・Do)	4
2.4	第 4 回: 過去成績の追加 (Do)	4
2.5	第 5 回: 過去成績の相対化 (Do・Check)	4
2.6	第 6 回: 回収率による評価 (Do)	5
2.7	第 7 回: ハイパーパラメータ調整 (Act)	5
2.8	実験目的	5
2.9	実験手順	5
2.10	データセット	6
2.11	モデル	6
2.12	パラメータ調整	7
2.13	方針	7
2.14	木の木数 (n_estimators)	7
2.15	木の深さ (max_depth)	7
2.16	learning_rate	7
2.17	scale_pos_weight	7
2.18	L2 正則化 (reg_lambda)	7
2.19	L1 正則化 (reg_alpha)	8
3	実験	8
3.1	使用した特徴量	8
3.2	実験結果	9
3.3	木の深さ (max_depth)	14
3.4	learning_rate	16
3.5	scale_pos_weight	18
3.6	L2 正則化 (reg_lambda)	19
3.7	L1 正則化 (reg_alpha)	21
4	最終的な設定まとめ	22

5	考察	23
6	意図していた実行計画との違い	23
7	まとめ	24

1 本実験向け補足

1.1 競馬複勝予測とは

本グループでは、競馬における馬の複勝率予測を対象問題として設定した。複勝とは、出走した馬が3着以内に入るかどうかを予測する馬券の種類であり、単勝や馬連と比較して的中しやすい特徴を持つ。本テーマでは、過去の競馬データを用いて、各馬が複勝するか否かを機械学習によって分類することを目的とする。競馬では、馬の年齢や体重、レース距離、天候、馬場状態など多くの要因がレース結果に影響すると考えられている。しかし、人間がこれらの膨大な情報を総合的に判断して予測を行うことは容易ではない。そこで、機械学習を活用することで、多数の説明変数から複勝に関係する特徴を学習し、客観的な予測を行うことが可能となる。

本テーマに取り組む意義は、実際の競馬予想への応用だけでなく、多数の特徴量を持つ実データに対して分類モデルを適用し、その性能や特徴量の重要度を分析できる点にある。これにより、機械学習を用いた予測手法の有効性を検証するとともに、データ分析の実践的な知識を深めることができる。

1.2 実験計画

2 実験計画と改善（PDCA）

本研究では、一度モデルを作成して終わりではなく、実験結果を分析し、改善を繰り返すPDCAサイクルを意識して開発を進めた。各回の実験内容と、それを踏まえた改善内容を以下に示す。

2.1 第1回：ベースラインモデルの構築（Plan・Do）

まず、競馬データをダウンロードし、最低限の特徴量として「馬体重」「天候」「馬齢」「距離」「芝・ダート区分」「着順」を抽出した。また、着順が3着以内であれば複勝フラグを1、それ以外を0として教師データを作成した。

その後、ロジスティック回帰、Random Forest、XGBoostの3種類のモデルを実装し、Accuracy、F1-score、ROC-AUC、Precision、Recallの5つの指標で性能評価を行った。さらに、特徴量重要度を確認し、予測に大きく影響する特徴量を分析した。

- **Check**：基本的な学習・評価環境を構築できたが、特徴量が少なく予測性能には改善の余地があることが分かった。
- **Action**：予測性能向上のため、新たな特徴量を追加することにした。

2.2 第2回：人気・単勝オッズの追加（Do・Check）

特徴量として「人気」と「単勝オッズ」を追加し、再度学習・評価を行った。

- **Check**：モデル性能は向上したが、人気や単勝オッズは人の予想が反映された情報であり、本来予測したい馬自身の能力とは異なる情報に強く依存していることが分かった。
- **Action**：人の判断が介在する特徴量を除外し、馬自身の能力を表す特徴量のみで予測できるモデルを目指すことにした。

2.3 第3回：特徴量の見直し（Plan・Do）

人気と単勝オッズを削除し、生データに含まれる特徴量のみを用いてモデルを再構築した。また、生データに存在する他の特徴量も追加し、情報量の増加を図った。

- **Check**：人の予想情報に依存しないモデルとなったが、単一レースの情報だけでは馬の実力を十分に表現できないことが分かった。
- **Action**：各馬の過去成績を特徴量として追加することにした。

2.4 第4回：過去成績の追加（Do）

馬ごとの過去成績を特徴量として追加した。具体的には、過去の平均着順や複勝率などを計算し、現在のレース予測に利用した。

- **Check**：過去成績を追加することで、馬の実力をより適切に表現できるようになった。しかし、絶対値だけではレース内での相対的な強さを十分に表現できていないことが分かった。
- **Action**：同一レース内で比較しやすい特徴量へ変換することにした。

2.5 第5回：過去成績の相対化（Do・Check）

過去成績を同一レース内で比較できるように加工した。具体的には、過去成績をレースごとに標準化するとともに、順位付けを行い、0から1の範囲に正規化した特徴量を作成した。

- **Check**：各馬の実力を同一レース内で比較しやすくなり、特徴量の表現力が向上した。
- **Action**：モデル性能だけでなく、実際の馬券購入を想定した評価指標も導入することにした。

2.6 第6回：回収率による評価（Do）

Accuracy や ROC-AUC などの機械学習指標に加え、各モデルについて馬券の回収率を算出し、実運用を意識した評価を行った。

- **Check**：分類性能が高いモデルと、実際の回収率が高いモデルは必ずしも一致しないことが分かった。
- **Action**：回収率も考慮しながら、モデル性能をさらに改善するため、ハイパーパラメータ調整を実施することにした。

2.7 第7回：ハイパーパラメータ調整（Act）

各モデルについてハイパーパラメータを調整し、デフォルト設定との性能比較を行った。

- **Check**：パラメータ調整による性能向上の効果を確認し、各モデルの特性や最適な設定について理解を深めることができた。
- **Action**：今後は特徴量のさらなる改善や特徴量選択、アンサンブル手法なども検討し、回収率を含めた総合的な性能向上を目指す。

2.8 実験目的

本実験の目的は、競馬の過去データを用いて機械学習モデルを構築し、出走馬が複勝（3着以内に入着）するかどうかを予測できるかを検証することである。競馬のレース結果には、馬齢、馬体重、距離、天候、馬場状態など多くの要因が関係していると考えられるため、これらの特徴量から複勝の可否を予測する分類モデルを作成する。また、複数の特徴量を用いた機械学習によってどの程度の予測性能が得られるのかを評価するとともに、どの特徴量が複勝予測に大きく影響しているのかを分析することも目的とする。さらに、異なる機械学習手法や前処理方法を比較し、それらが予測精度に与える影響を検証する。これらの実験を通して、競馬における複勝予測の実現可能性を確認するとともに、実データを対象とした機械学習モデル構築の有効性について考察する。

2.9 実験手順

第三者が実験を再現するための手順を以下に示す。

1. GitHub リポジトリを取得する。

```
git clone <リポジトリ URL>
cd horse_racing
```

2. 必要なライブラリをインストールする.

```
uv sync
```

3. Kaggle から以下のファイルをダウンロードし, data ディレクトリに配置する.

- 19860105-20210731_race_result.csv
- 19860105-20210731_odds.csv

4. 再現したい実験に対応するコミットへ移動する.

```
git checkout <コミット ID>
```

5. コミットに対応した実験を実行する.

- 最終コミット版

```
uv run python Preprocess/make_features.py
```

```
uv run python learn_model/XGBoost.py --max_depth 7 --learning_rate 0.07
```

```
uv run python evaluation/evaluate_models.py
```

- 最終一歩手前のコミット版

```
./run_all.sh
```

2.10 データセット

https://www.kaggle.com/datasets/takamotoki/jra-horse-racing-dataset?select=19860105-20210731_race_result.csv

本実験では, 以下の 2 つのファイルを利用する.

1. 19860105-20210731_race_result.csv

各レースに出走した馬ごとの情報が記録されているファイルである. 馬体重, 馬体重増減, 馬齢, 競馬場, 距離, 天候, 馬場状態, 騎手など, 多数の説明変数が含まれている.

2. 19860105-20210731_odds.csv

各レースのオッズ情報が記録されているファイルである. 本研究では単勝オッズなどの情報を取得するために利用した.

2.11 モデル

- ロジスティック回帰
- XGBoost
- RandomForest

2.12 パラメータ調整

2.13 方針

時間的制約から、パラメータ調整は XGBoost のみを対象とした。調整は 1 項目ずつ値を振り、分類指標 (Accuracy, F1, ROC-AUC, Precision, Recall) と複勝回収率 (モデル 1 位・人気 1 位・モデル上位 3 頭・人気上位 3 頭) の変化をグラフで確認しながら、最善と判断した値を次の項目のベースラインに固定して進める、座標降下的な手順を取った。項目の順序は、木の本数 → 木の深さ → learning_rate → scale_pos_weight → L2 正則化 → L1 正則化である。

評価にあたっては、回収率の絶対的な最大値だけでなく、AUC・F1・Recall とのトレードオフを重視した。回収率がほぼ変わらないにもかかわらず分類指標 (特に Recall) が大きく悪化する場合は、過学習のサインとみなし、より保守的な値を採用する方針とした。具体的な結果・考察・採用値は次の「実験結果」章にまとめる。

2.14 木の本数 (n_estimators)

100 → 1000 に変更した。木の本数を増やすこと自体に悪影響はないと判断し、個別の検証・グラフ化は行わなかった。

2.15 木の深さ (max_depth)

候補として 4~10 を設定し、learning_rate はデフォルト値 (0.1) に固定して実施した。

2.16 learning_rate

候補は 0.1 以下 (0.01, 0.03, 0.05, 0.07, 0.1) とした。初回は max_depth の調整結果を反映する前だったため、暫定的に depth=10 で実施した。しかし depth と learning_rate には交互作用があり、片方を固定したまま他方だけを見て結論を出すのは適切でないと判断し、max_depth=7 を確定させたうえで再度スweepを行った。

2.17 scale_pos_weight

候補として auto(neg/pos, 既定)、1.5、1.2、1.0 を設定し、max_depth=7、learning_rate=0.07 で実施した。

2.18 L2 正則化 (reg_lambda)

候補として 0, 0.5, 1, 5 を設定し、max_depth=7、learning_rate=0.07、scale_pos_weight=auto で実施した。

2.19 L1 正則化 (reg_alpha)

候補として 0, 0.5, 1, 5 を設定し、max_depth=7、learning_rate=0.07、scale_pos_weight=auto で実施した。

3 実験

3.1 使用した特徴量

表 1 基本特徴量

特徴量名	説明
馬齢	競走馬の年齢
馬体重	レース当日の馬体重
馬体重増減	前走からの馬体重の増減
斤量	負担重量
性別	牡・牝・セン
距離	レース距離 (m)
芝・ダート区分	芝コースかダートコースか
枠番	出走時の枠番号

表 2 レース条件に関する特徴量

特徴量名	説明
競馬場名	開催競馬場
馬場状態	良・稍重・重・不良
天候	レース当日の天候
右左回り・直線区分	コースの回り方向
距離帯	距離を 5 区分にカテゴリ化 (短距離・マイル・中距離・中長距離・長距離)
レースクラス	新馬・未勝利・1~3 勝クラス・オープンの 6 区分
レースグレード	G1・G2・G3・L・なしの 5 区分
休養日数	前走からの日数

表 3 過去成績に基づく特徴量

特徴量名	説明
過去 3 走・5 走平均着順	直近の平均着順
過去 3 走・5 走複勝率	直近の複勝 (3 着以内) 率
過去 3 走・5 走平均上り	直近のラスト 3F 平均タイム
騎手複勝率	騎手の通算複勝率
調教師複勝率	調教師の通算複勝率
芝・ダート別複勝率	芝/ダート別の複勝率
距離帯別複勝率	距離帯別の複勝率
競馬場別複勝率	競馬場別の複勝率

すべての過去成績に基づく特徴量は、対象レースより過去のデータのみを用いて算出しており、未来の情報が混入しないよう `shift` 処理を施している。

表 4 同一レース内での相対化特徴量

特徴量名	説明
順位率	同一レース内でのパーセンタイル順位 (0 に近いほど上位)
標準化	同一レース内での標準化得点 (Z スコア)

上記の過去成績特徴量および馬体重・斤量・馬齢・休養日数について、同一レースに出走する馬同士で比較した相対値を追加で作成した。

3.2 実験結果

3.2.1 実験 1: ベースライン

使用した特徴量: 馬齢, 馬体重, 距離 (m), 芝・ダート区分

表 5 実験 1 の結果

モデル	Accuracy	F1	ROC-AUC	Precision	Recall
XGBoost	0.4794	0.3544	0.5595	0.2427	0.6565
Random Forest	0.5012	0.3363	0.5408	0.2367	0.5804
ロジスティック回帰	0.4637	0.3544	0.5569	0.2401	0.6762

3.2.2 実験 2: 市場予想 (単勝オッズ・人気) の追加

実験 1 の特徴量に加え、単勝オッズ・人気を追加した。

表 6 実験 2 の結果

モデル	Accuracy	F1	ROC-AUC	Precision	Recall
XGBoost	0.7390	0.5456	0.8100	0.4393	0.7196
Random Forest	0.7551	0.4730	0.7611	0.4449	0.5048
ロジスティック回帰	0.7101	0.5368	0.8068	0.4115	0.7719

3.2.3 実験 3: 市場予想を除外した特徴量

単勝オッズ・人気を除外し、代わりに斤量, 馬体重増減, 枠番, 休養日数, 性別, 競馬場名, 馬場状態, 天候, 右左回り・直線区分, 距離帯, レースクラス, レースグレードを追加した。

表 7 実験 3 の結果

モデル	Accuracy	F1	ROC-AUC	Precision	Recall
XGBoost	0.5448	0.3724	0.6009	0.2660	0.6203
Random Forest	0.7145	0.2233	0.5655	0.2737	0.1885
ロジスティック回帰	0.4681	0.3660	0.5770	0.2471	0.7053

3.2.4 実験 4: 過去成績特徴量の追加

実験 3 の特徴量に加え、過去平均着順, 過去複勝率, 調教師複勝率, 騎手複勝率, 距離帯別複勝率, 過去平均上り, 芝・ダート別複勝率, 競馬場別複勝率を追加した。

表 8 実験 4 の結果

モデル	Accuracy	F1	ROC-AUC	Precision	Recall
XGBoost	0.7069	0.4806	0.7449	0.3913	0.6230
Random Forest	0.7712	0.4106	0.7356	0.4674	0.3662
ロジスティック回帰	0.7025	0.4667	0.7295	0.3827	0.5981

3.2.5 実験 5: 同一レース内での相対化

過去成績や馬体重などの特徴量について、同一レース内での順位率・標準化を追加した。

表 9 実験 5 の結果

モデル	Accuracy	F1	ROC-AUC	Precision	Recall
XGBoost	0.7110	0.4957	0.7612	0.3997	0.6524
Random Forest	0.7761	0.4358	0.7525	0.4825	0.3973
ロジスティック回帰	0.7028	0.4864	0.7492	0.3898	0.6466

3.2.6 実験 6: 回収率による評価

実験 5 のモデルを用いて、閾値方式による複勝回収率を評価した。実験 5 のモデルを用いて、モデルの予測確率が一定の閾値を超えた場合にのみ馬券を購入したと仮定し、複勝回収率を評価した。購入方法として、以下の 2 種類を比較した。

- **全頭方式**: 閾値を超えた馬を、1 レースにつき複数頭でもすべて購入する方式 (該当馬が 0 頭のレースは購入しない)。
- **1 位方式**: 各レースで予測確率が最も高い馬のみを対象とし、その馬の予測確率が閾値を超えている場合にのみ購入する方式 (1 レースにつき購入は 0 頭または 1 頭のみ)。

いずれの方式も、1 頭 (1 点) あたり 100 円を購入したものと仮定し、以下の式で回収率を算出した。

$$\text{回収率} = \frac{\text{総払戻金}}{\text{総購入金額}} \times 100$$

なお、閾値を高く設定するほど購入対象となる頭数 (レース数) は減少するため、回収率とあわせて実際に賭けた頭数・購入レース数も併記する。

XGBoost

表 10 に全頭方式、表 11 に 1 位方式の結果を示す。

表 10 実験 6:XGBoost 回収率 (閾値&全頭方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	80.05	80.43	81.17	82.84	85.65
賭けた頭数	475,808	317,233	177,845	68,606	8,332

表 11 実験 6:XGBoost 回収率 (閾値&1 位方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	81.16	81.24	81.47	82.70	85.61
購入レース数	97,535	96,248	86,245	51,705	8,081

Random Forest

表 12 に全頭方式、表 13 に 1 位方式の結果を示す。

表 12 実験 6:Random Forest 回収率 (閾値&全頭方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	80.09	80.91	82.66	83.83	89.44
賭けた頭数	240,000	109,079	37,422	7,935	557

表 13 実験 6:Random Forest 回収率 (閾値&1 位方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	80.28	80.90	82.40	83.70	89.39
購入レース数	93,657	68,701	31,154	7,527	555

ロジスティック回帰

表 14 に全頭方式、表 15 に 1 位方式の結果を示す。

表 14 実験 6: ロジスティック回帰 回収率 (閾値&全頭方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	79.33	79.31	80.09	81.19	84.39
賭けた頭数	483,452	299,497	160,847	62,786	8,634

表 15 実験 6: ロジスティック回帰 回収率 (閾値&1 位方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	79.81	79.91	80.36	81.18	84.43
購入レース数	97,572	95,383	84,069	51,026	8,539

3.2.7 参考: 単勝オッズ・人気を特徴量に含めたモデル

実験 5 の特徴量に加えて単勝オッズおよび人気を追加したモデルについても、同様の方法で複勝回収率を算出した。なお、これらのモデルは人気 (市場予想) そのものを特徴量として利用しているため、モデルの予測が人気の情報を強く反映しやすい点に留意されたい。

■XGBoost 表 16 に全頭方式、表 17 に 1 位方式の結果を示す。

表 16 参考:XGBoost(オッズ・人気を含む) 回収率 (閾値&全頭方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	82.07	82.58	83.70	85.64	88.61
賭けた頭数	473,021	343,980	219,883	106,985	25,104

表 17 参考:XGBoost(オッズ・人気を含む) 回収率 (閾値&1 位方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	84.15	84.15	84.21	85.31	88.66
購入レース数	97,595	97,571	96,330	77,999	24,759

■Random Forest 表 18 に全頭方式、表 19 に 1 位方式の結果を示す。

表 18 参考:Random Forest(オッズ・人気を含む) 回収率 (閾値&全頭方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	82.33	83.66	85.19	86.90	90.18
賭けた頭数	283,057	164,133	76,504	23,721	2,754

表 19 参考:Random Forest(オッズ・人気を含む) 回収率 (閾値&1 位方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	83.63	83.87	85.01	86.89	90.10
購入レース数	97,403	90,379	60,840	22,644	2,752

■ロジスティック回帰 表 20 に全頭方式、表 21 に 1 位方式の結果を示す。

表 20 参考: ロジスティック回帰 (オッズ・人気を含む) 回収率 (閾値&全頭方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	80.63	81.17	82.16	84.06	91.43
賭けた頭数	528,115	393,795	244,212	74,233	91

表 21 参考: ロジスティック回帰 (オッズ・人気を含む) 回収率 (閾値&1 位方式)

閾値	0.5	0.6	0.7	0.8	0.9
回収率 (%)	83.09	83.09	83.09	84.03	91.43
購入レース数	97,595	97,595	97,559	66,058	91

3.2.8 実験 7: パラメータ調整

各パラメータのスイープ結果を以下に示す。n_estimators は検証を行っていないため結果は掲載しない。

3.3 木の深さ (max_depth)

表 22 max_depth スイープ結果

値	Accuracy	F1	ROC-AUC	Precision	Recall	1 位回収率	上位 3 頭回収率
4	.7411	.5469	.8117	.4417	.7177	84.63	82.79
5	.7439	.5460	.8100	.4446	.7073	84.65	83.07
6	.7479	.5442	.8074	.4487	.6914	84.86	83.17
7	.7538	.5403	.8033	.4551	.6645	84.95	83.17
8	.7611	.5355	.7994	.4642	.6325	84.87	83.25
9	.7700	.5282	.7952	.4772	.5915	84.97	83.25
10	.7781	.5192	.7918	.4914	.5503	84.84	83.41

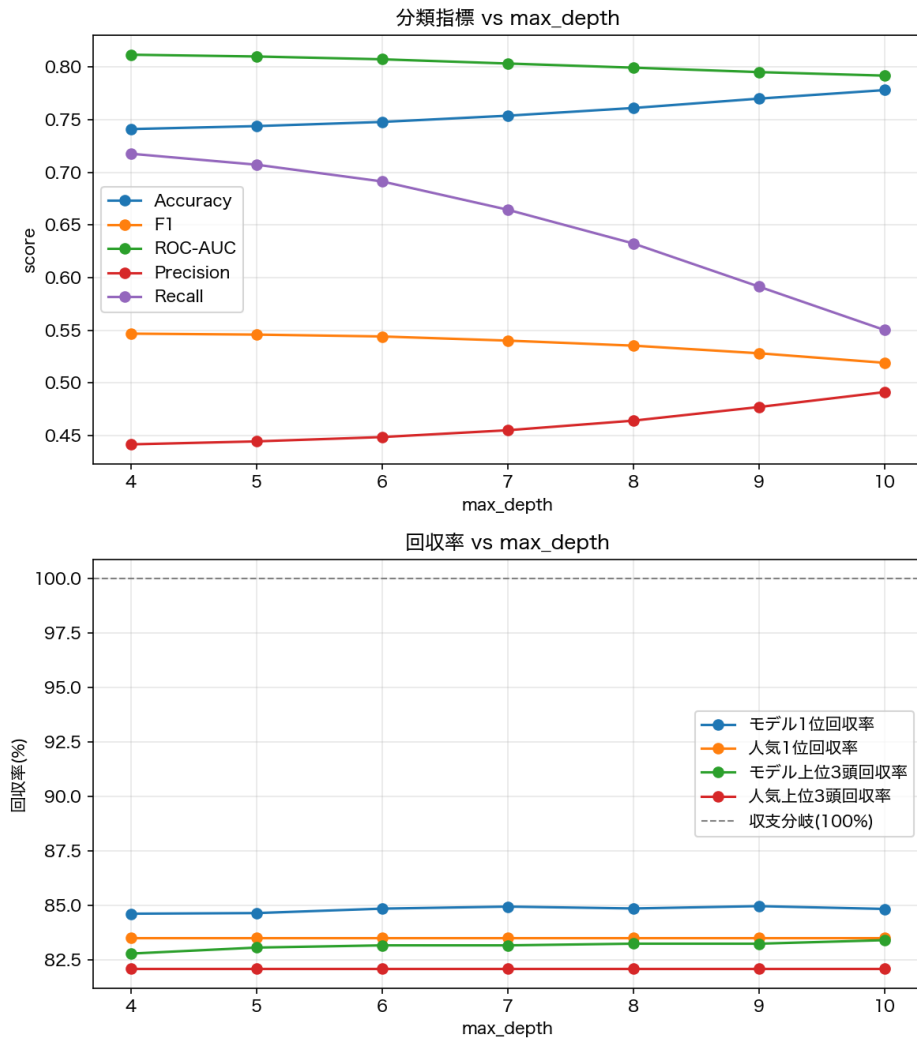


図1 分類指標・回収率 vs max_depth (learning_rate=0.1)

■**考察** 深さを増やすほど AUC(.812→.792)・F1(.547→.519)・Recall(.718→.550) が単調に悪化する一方、回収率は depth=6 付近でほぼ頭打ちになり、6→10 まで増やしてもモデル 1 位回収率は +0.1pt 未満、モデル上位 3 頭回収率でも +0.2pt 程度しか伸びない。回収率がほとんど変わらないのに判別力と Recall だけ大きく犠牲になっており、depth=9・10 は過学習方向に寄りすぎていると判断した。

■**採用値** max_depth = 7。回収率をほぼ落とさずに分類指標を保てる境界として採用した。

3.4 learning_rate

3.4.1 スイープ結果 (max_depth=7、採用)

表 23 learning_rate スイープ結果 (max_depth=7、採用)

値	Accuracy	F1	ROC-AUC	Precision	Recall	1 位回収率	上位 3 頭回収率
0.01	.7413	.5477	.8127	.4421	.7197	84.19	82.53
0.03	.7443	.5469	.8113	.4451	.7089	84.59	82.81
0.05	.7477	.5458	.8094	.4488	.6964	84.75	82.91
0.07	.7505	.5440	.8071	.4518	.6837	84.81	83.18
0.10	.7538	.5403	.8033	.4551	.6645	84.95	83.17

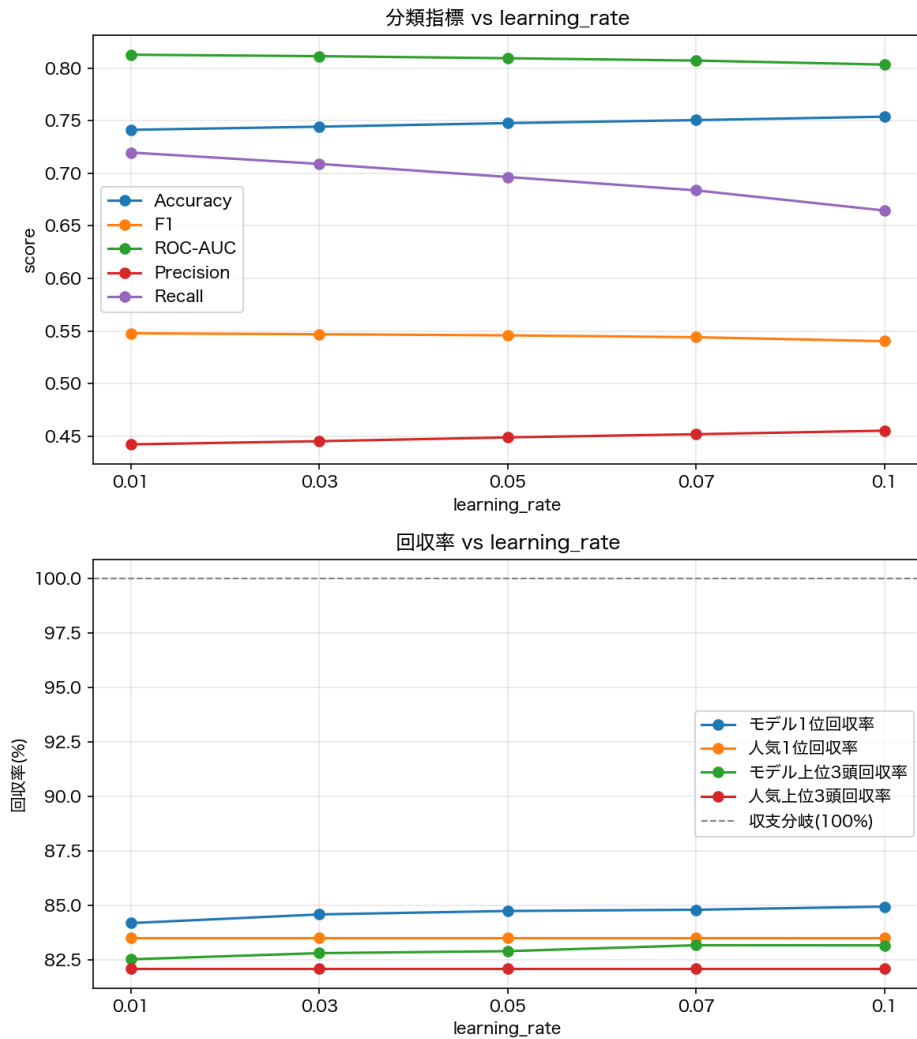


図2 分類指標・回収率 vs learning_rate (max_depth=7、採用)

■**考察** depth=7 では rate=0.05 のピークは再現されず、モデル1位回収率は rate=0.1 に向けて単調に増加した。rate=0.07 と 0.1 を比較すると、モデル1位回収率は 0.1 がわずかに上 (+0.15pt) だが、モデル上位3頭回収率は 0.07 がわずかに上 (実質同着)。AUC(.8071 vs .8033)・F1(.5440 vs .5403)・Recall(.6837 vs .6645、+1.9pt) はいずれも 0.07 が明確に上回る。0.1 は回収率のわずかな上乗せと引き換えに分類指標を明確に犠牲にする選択であり、depth のときと同じ構図であるため 0.07 を採用した。

■**採用値** learning_rate = 0.07。

表 24 scale_pos_weight スイープ結果

値	Accuracy	F1	ROC-AUC	Precision	Recall	1 位回収率	上位 3 頭回収率
auto (neg/pos)	.7505	.5440	.8071	.4518	.6837	84.81	83.18
1.5	.8029	.5038	.8095	.5575	.4595	85.01	83.00
1.2	.8084	.4674	.8098	.5916	.3863	84.73	83.01
1.0	.8101	.4308	.8099	.6199	.3301	85.04	83.06

3.5 scale_pos_weight

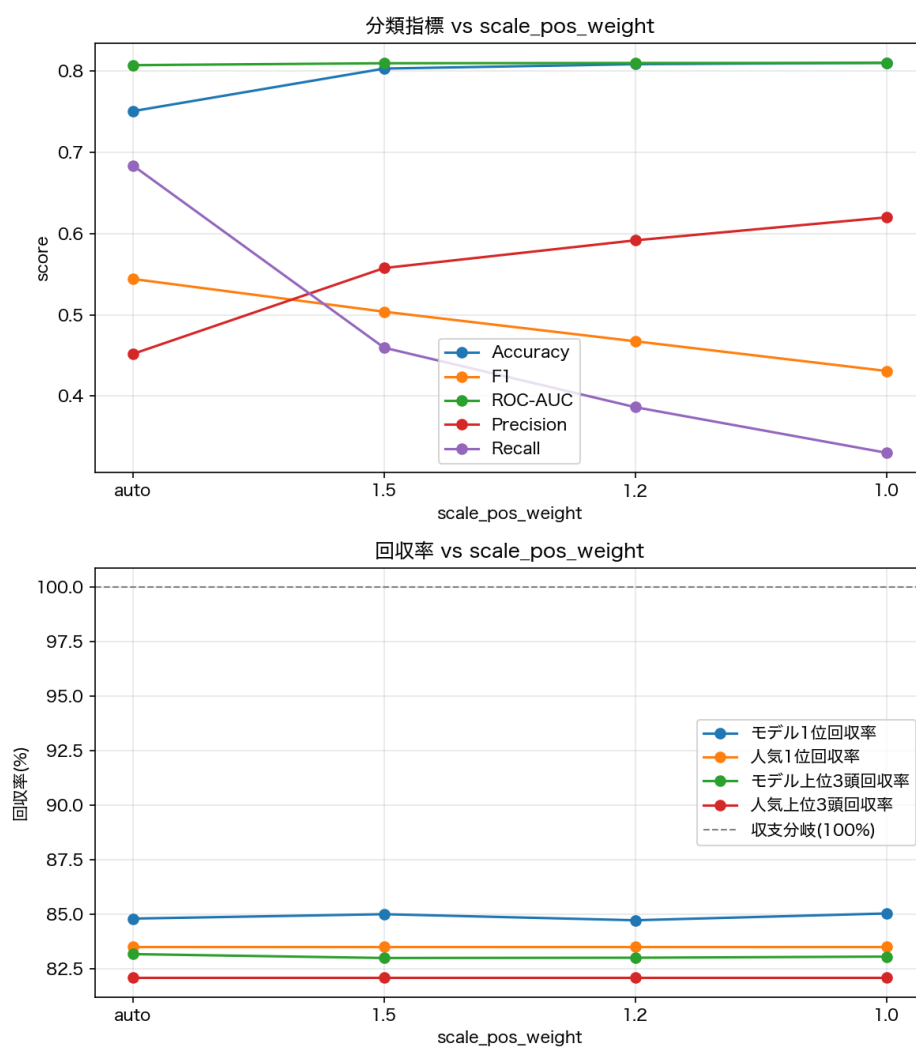


図 3 分類指標・回収率 vs scale_pos_weight (max_depth=7, learning_rate=0.07)

■考察 AUC は.807～.810 とほぼ横並びでこの項目への感度が低い。一方で F1・Recall は auto から離れるほど急激に悪化し (Recall は.684→.330 と半減以下)、Precision は逆に大きく上昇する (.45→.62)。これは scale_pos_weight を下げるほどモデルが確信度の高い馬だけに予測を絞り込むためである。回収率で見ても auto は大きく劣らず、モデル上位 3 頭回収率は auto が最高 (83.18%)。モデル 1 位回収率は 1.0 がわずかに最高 (85.04%) だが 1.2 は auto より低い (84.73%) という非単調な動きで、誤差の範囲とみられる。

■採用値 scale_pos_weight = auto (neg/pos)。固定値は回収率をほぼ変えずに Recall を大きく犠牲にするため、既定の auto を維持した。

3.6 L2 正則化 (reg_lambda)

表 25 reg_lambda スイープ結果

値	Accuracy	F1	ROC-AUC	Precision	Recall	1 位回収率	上位 3 頭回収率
0.0	.7499	.5427	.8064	.4508	.6818	84.76	83.18
0.5	.7503	.5433	.8068	.4514	.6823	84.73	83.17
1.0	.7505	.5440	.8071	.4518	.6837	84.81	83.18
5.0	.7501	.5450	.8081	.4514	.6876	84.73	83.07

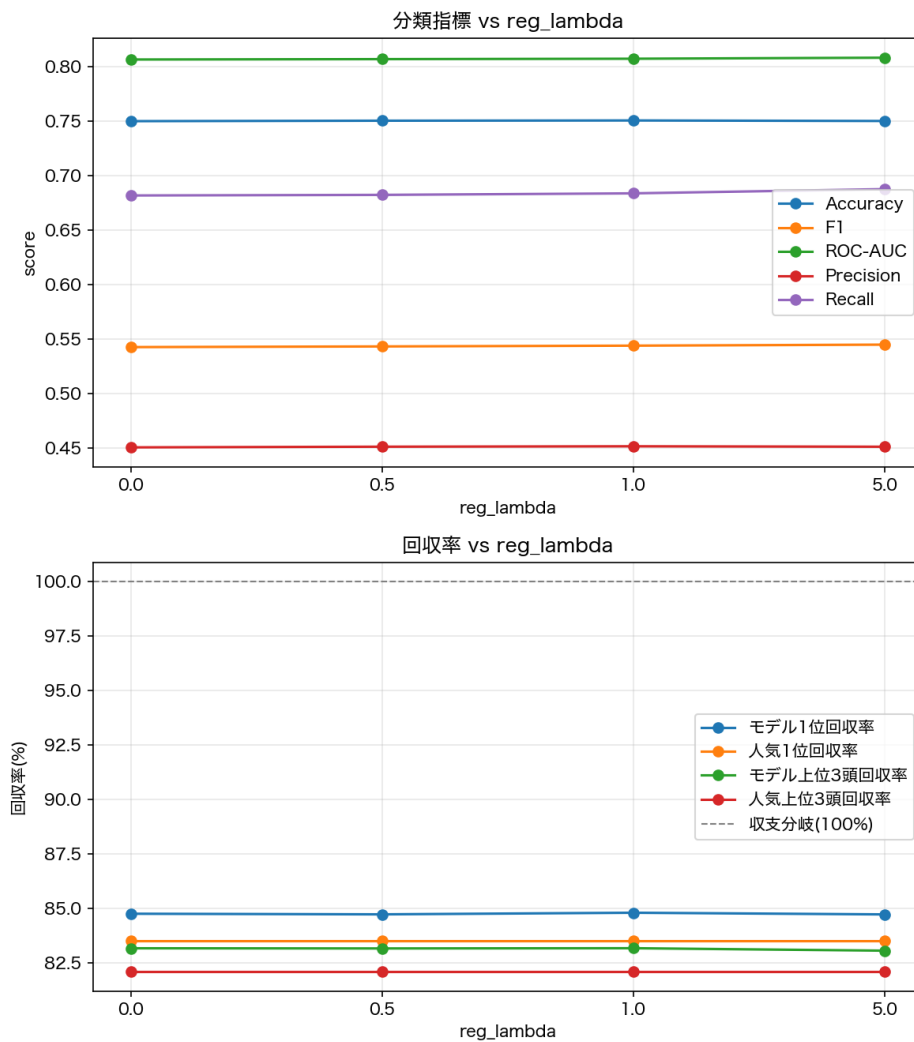


図4 分類指標・回収率 vs reg_lambda (max_depth=7, learning_rate=0.07, scale_pos_weight=auto)

■考察 AUC(.8064~.8081)・F1(.5427~.5450)・回収率(いずれも0.1pt程度の範囲)とも、0~5の範囲でほぼ横ばいだった。以前の検証で「正則化はほとんど効果がない」とされていた所見と一致する結果となった。

■採用値 reg_lambda = 1.0 (既定値)。差が誤差レベルであり、変更する積極的な理由がないため既定値を維持した。

3.7 L1 正則化 (reg_alpha)

表 26 reg_alpha スイープ結果

値	Accuracy	F1	ROC-AUC	Precision	Recall	1 位回収率	上位 3 頭回収率
0.0	.7505	.5440	.8071	.4518	.6837	84.81	83.18
0.5	.7506	.5437	.8070	.4518	.6824	84.85	83.09
1.0	.7509	.5439	.8071	.4522	.6824	84.76	83.13
5.0	.7512	.5448	.8081	.4527	.6841	85.09	83.22

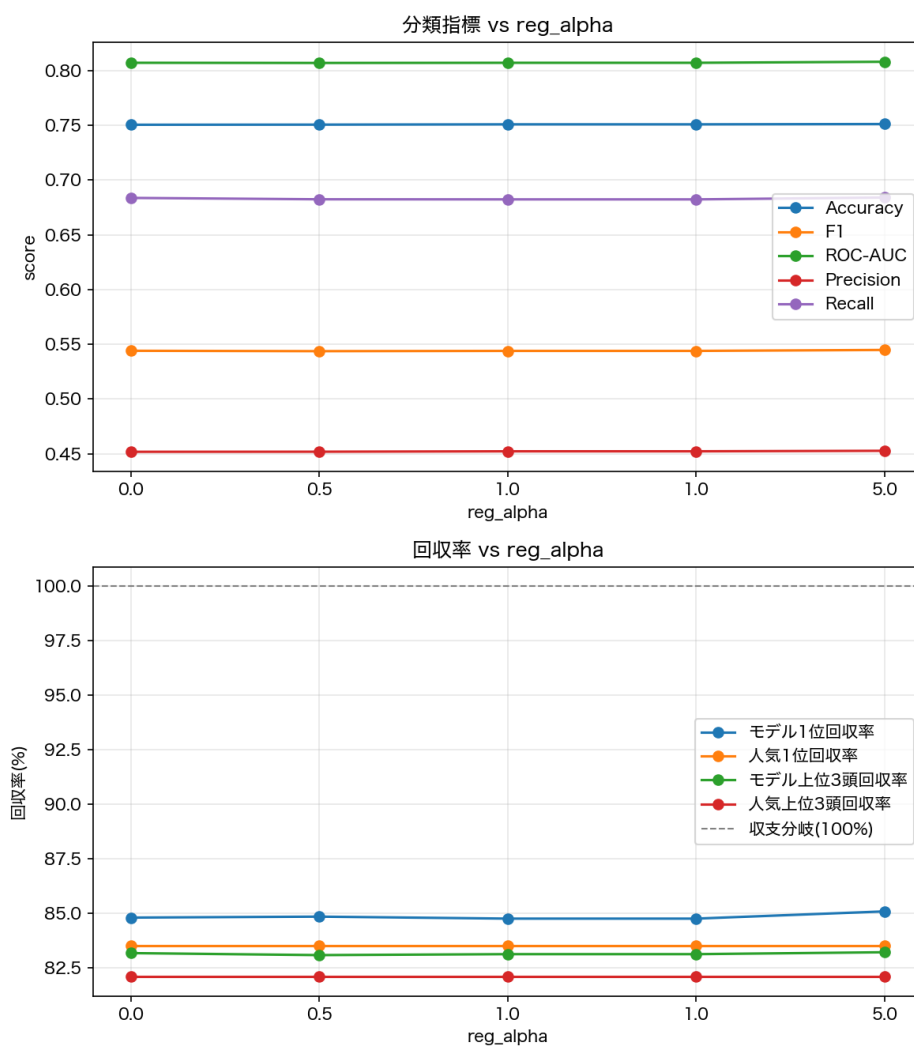


図 5 分類指標・回収率 vs reg_alpha (max_depth=7, learning_rate=0.07, scale_pos_weight=auto)

■**考察** reg_alpha=0 の行は reg_lambda スイープ時の基準値 (depth=7, rate=0.07, scale_pos_weight=auto, reg_lambda=1) と完全に一致しており、条件の整合性を確認済み。AUC(.8070～.8081)・F1(.5437～.5448)・回収率 (1 位回収率で 0.3pt 程度、上位 3 頭回収率で 0.13pt 程度の範囲) とも、0～5 の範囲でほぼ横ばいであり、reg_lambda と同様に L1 正則化の効果もほぼ見られなかった。reg_alpha=5.0 が AUC・F1・Recall・回収率のいずれもわずかに最良だが、差は誤差レベルである。

■**採用値** reg_alpha = 0 (既定値)。reg_alpha=5.0 がわずかに全指標で上回るが、差はごくわずかでありモデルを複雑にする積極的な理由がないため、既定値を維持した。

4 最終的な設定まとめ

表 27 最終的な XGBoost のハイパーパラメータ設定

パラメータ	採用値	根拠 (要約)
n_estimators	1000	デフォルトの 100 から増加。過学習リスクなしと判断し検証は省略。
max_depth	7	回収率は depth=6 以降ほぼ頭打ちなのに対し、AUC/F1/Recall は深くするほど悪化。回収率をほぼ落とさず分類指標を保てる境界として採用。
learning_rate	0.07	depth=10 で行った初回スイープでは rate=0.05 が回収率のピークだったが、depth=7 で再スイープすると傾向が変化。0.1 は 1 位回収率が最大だが AUC/F1/Recall を明確に犠牲にするため、回収率をほぼ落とさず分類指標を保てる 0.07 を採用。
scale_pos_weight	auto (neg/pos)	固定値 (1.5, 1.2, 1.0) は AUC にほぼ影響しないが F1・Recall を大きく悪化させ、上位 3 頭回収率も低下する。既定の auto(fold 毎に neg/pos を計算) を維持。
reg_lambda (L2)	1.0 (既定値)	0～5 の範囲で全指標がほぼ横ばい。効果は無視できる水準のため既定値を維持。
reg_alpha (L1)	0 (既定値)	reg_lambda と同様、0～5 の範囲でほぼ横ばい (reg_alpha=0 の基準値は reg_lambda スイープ時と完全に一致し条件の整合性を確認済み)。効果は無視できる水準のため既定値を維持。

5 考察

03:13 kai 第4回実験では、過去3走・5走の平均着順、複勝率、上りタイム、騎手複勝率、調教師複勝率などの過去情報を特徴量として追加した。その結果、第3回実験と比較して各モデルの性能が向上した。特に XGBoost では ROC-AUC が 0.6009 から 0.7449 へ向上した。これは、競走馬の現在の状態だけでなく、過去の成績や得意条件などの傾向が複勝予測に有効であったためだと考えられる。

第5回実験では、既存の特徴量を同一レース内で相対化した特徴量を追加した。その結果、XGBoost では ROC-AUC が 0.7449 から 0.7612 へ向上した。競馬では、馬体重や過去成績などの絶対値だけではなく、同じレースに出走する馬との相対的な能力差も結果に影響すると考えられる。そのため、レース内での位置づけを表す特徴量が有効に働いたと考えられる。

回収率評価では、予測確率の閾値を高く設定するほど回収率が向上する傾向が確認された。これは、予測確率が高い馬に限定することで、モデルが複勝と判断した馬の中でも、より確信度の高い馬を選択できたためだと考えられる。一方で、閾値を高くすると購入対象となる馬が減少するため、安定した収益を得るためには回収率だけでなく購入機会も考慮する必要がある。

モデルごとに性能の傾向に違いが見られた。XGBoost は ROC-AUC が高く、予測確率の順位付け性能に優れていた。一方で、回収率評価では Random Forest が最も高い結果となった。このことから、分類性能が高いモデルが必ずしも収益性の高いモデルになるとは限らず、実際の購入戦略を考慮した評価が重要である。

パラメータ調整では大きな性能向上は確認できなかった。このことから、今回使用したデータではモデルのパラメータ調整よりも、入力する特徴量の設計が性能に大きな影響を与えたと考えられる。03:13 kai 考察です。チャッピーに書かせました 03:15 kai 今回使用した特徴量は、過去成績や騎手・調教師の実績などを中心としており、展開予測や血統、馬場適性などの情報は考慮していない。そのため、これらの情報を追加することで、さらなる予測性能の向上が期待できる。また、今回は複勝を対象とした二値分類モデルを構築したが、購入金額の最適化や賭け方の戦略については検討していない。今後は、より多様な特徴量の追加や購入戦略の改善を行い、予測性能だけでなく実際の回収率向上を目指す必要がある。

6 意図していた実行計画との違い

当初の実験計画では、モデルの良し悪しを Accuracy・F1・ROC-AUC・Precision・Recall といった機械学習上の分類指標のみで評価し、特徴量を追加するごとにこれらの指標が向上することを目的としていた(第1~5回実験)。しかし、実験5終了時点で分類指標を確認した際、例えば ROC-AUC が 0.75 から 0.76 に上がった、F1 が 0.49 であるといった数値だけでは、そのモデルが複勝予測として実用上「良い」のかどうかを判断することが難しいという問題に直面した。分類指標はあくまでモデルの統計的な性能を示すものであり、実際に馬券を購入した場合にどの程度の収益が見込めるかという、本来知りたい実用的な価値とは直接結びついていなかったためである。

そこで、当初の計画にはなかった評価軸として、第 6 回実験より複勝回収率 (実際に馬券を購入したと仮定した場合の払戻金÷購入金額) を新たに導入した。回収率は 100

この変更により、分類指標では優れていたモデル (XGBoost) が、回収率では必ずしも最良ではない (場面によっては Random Forest が同等以上の回収率を示す) という、分類指標だけを見ていては気づけなかった知見が得られた。結果として、以降のパラメータ調整 (第 7 回実験) においても、分類指標と回収率の両方を確認しながらトレードオフを判断する方針に変更している。すなわち、当初は「予測精度 (分類指標) の向上」を目的としていた実行計画が、実験を進める中で「予測精度と実用的な収益性 (回収率) を両立させる」という目的へと修正された点が、当初の計画との主な違いである。

7 まとめ

本研究では、競走馬の複勝予測を目的として、過去成績や騎手・調教師情報などを利用した機械学習モデルを構築した。基本的な競走馬情報のみを用いたモデルから始め、特徴量を段階的に追加することで、どのような情報が予測性能に影響するかを検証した。

実験の結果、過去の着順や複勝率、上りタイムなどの履歴情報を追加することで予測性能が向上し、競走馬の過去の傾向を利用することが複勝予測に有効であることが分かった。また、同一レース内で特徴量を相対化することで、出走馬間の比較情報をモデルに与えることができ、さらなる性能向上につながった。

本研究を通して、機械学習ではモデルの選択やパラメータ調整だけでなく、対象データに適した特徴量を設計することが重要であることを学んだ。また、競馬のような複雑な予測問題では、予測精度だけでなく、実際の利用目的に合わせた評価方法を考える必要があることを理解した。

参考文献

- [1] Kaggle, JRA 日本中央競馬会 Horse Racing Dataset, <https://www.kaggle.com/datasets/takamotoki/jra-horse-racing-dataset>, 参照日: 2026/07/28.